

Выделение и освобождение номеров устройств

```
1 extern int alloc_chrdev_region(dev_t *, unsigned, unsigned, const char *);
2 extern int register_chrdev_region(dev_t, unsigned, const char *);
3 extern int __register_chrdev(unsigned int major, unsigned int baseminor,
4                             unsigned int count, const char *name,
5                             const struct file_operations *fops);
6 extern void __unregister_chrdev(unsigned int major, unsigned int baseminor
7                                , unsigned int count, const char *name);
8 extern void unregister_chrdev_region(dev_t, unsigned);
9 extern void chrdev_show(struct seq_file *, off_t);
10
11 static inline int register_chrdev(unsigned int major, const char *name,
12                                   const struct file_operations *fops)
13 {
14     return __register_chrdev(major, 0, 256, name, fops);
15 }
16
17 static inline void unregister_chrdev(unsigned int major, const char *name)
18 {
19     __unregister_chrdev(major, 0, 256, name);
20 }
```

Листинг 1. Выделение и освобождение номеров устройств

Обработчики аппаратных прерываний

```
1 typedef irqreturn_t (*irq_handler_t)(int, void *);
2
3 static inline int __must_check
4 request_irq(unsigned int irq, irq_handler_t handler, unsigned long flags,
5             const char *name, void *dev)
6 {
7     return request_threaded_irq(irq, handler, NULL, flags |
8                                IRQF_COND_ONESHOT, name, dev);
9 }
```

Листинг 2. Обработчики аппаратных прерываний

Флаги IRQ

```
1 /*
2  * These flags used only by the kernel as part of the
3  * irq handling routines.
4  *
5  * IRQF_SHARED - allow sharing the irq among several devices
6  * IRQF_PROBE_SHARED - set by callers when they expect sharing mismatches
7  * to occur
8  * IRQF_TIMER - Flag to mark this interrupt as timer interrupt
9  * IRQF_PERCPU - Interrupt is per cpu
10  * IRQF_NOBALANCING - Flag to exclude this interrupt from irq balancing
11  * IRQF_IRQPOLL - Interrupt is used for polling (only the interrupt that
12  * is
13  *
14  * registered first in a shared interrupt is considered for
15  * performance reasons)
16  */
```

```

13 * IRQF_ONESHOT - Interrupt is not reenabled after the hardirq handler
    finished.
14 *
    Used by threaded interrupts which need to keep the
15 *
    irq line disabled until the threaded handler has been
    run.
16 * IRQF_NO_SUSPEND - Do not disable this IRQ during suspend. Does not
    guarantee
17 *
    that this interrupt will wake the system from a
    suspended
18 *
    state. See Documentation/power/suspend-and-
    interrupts.rst
19 * IRQF_FORCE_RESUME - Force enable it on resume even if IRQF_NO_SUSPEND
    is set
20 * IRQF_NO_THREAD - Interrupt cannot be threaded
21 * IRQF_EARLY_RESUME - Resume IRQ early during syscore instead of at
    device
    resume time.
22 *
23 * IRQF_COND_SUSPEND - If the IRQ is shared with a NO_SUSPEND user,
    execute this
24 *
    interrupt handler after suspending interrupts. For
    system
25 *
    wakeup devices users need to implement wakeup detection
    in
26 *
    their interrupt handlers.
27 * IRQF_NO_AUTOEN - Don't enable IRQ or NMI automatically when users
    request it.
28 *
    Users will enable it explicitly by enable_irq() or
    enable_nmi()
29 *
    later.
30 * IRQF_NO_DEBUG - Exclude from runaway detection for IPI and similar
    handlers,
31 *
    depends on IRQF_PERCPU.
32 * IRQF_COND_ONESHOT - Agree to do IRQF_ONESHOT if already set for a
    shared
33 *
    interrupt.
34 */
35 #define IRQF_SHARED 0x00000080
36 #define IRQF_PROBE_SHARED 0x00000100
37 #define IRQF_TIMER 0x00000200
38 #define IRQF_PERCPU 0x00000400
39 #define IRQF_NOBALANCING 0x00000800
40 #define IRQF_IRQPOLL 0x00001000
41 #define IRQF_ONESHOT 0x00002000
42 #define IRQF_NO_SUSPEND 0x00004000
43 #define IRQF_FORCE_RESUME 0x00008000
44 #define IRQF_NO_THREAD 0x00010000
45 #define IRQF_EARLY_RESUME 0x00020000
46 #define IRQF_COND_SUSPEND 0x00040000
47 #define IRQF_NO_AUTOEN 0x00080000
48 #define IRQF_NO_DEBUG 0x00100000
49 #define IRQF_COND_ONESHOT 0x00200000

```

Листинг 3. Флаги IRQ

struct irqaction

```

1 /**
2  * struct irqaction - per interrupt action descriptor
3  * @handler:      interrupt handler function

```

```

4  * @name:    name of the device
5  * @dev_id:  cookie to identify the device
6  * @percpu_dev_id:  cookie to identify the device
7  * @affinity:  CPUs this irqaction is allowed to run on
8  * @next:    pointer to the next irqaction for shared interrupts
9  * @irq:     interrupt number
10 * @flags:   flags (see IRQF_* above)
11 * @thread_fn:  interrupt handler function for threaded interrupts
12 * @thread:   thread pointer for threaded interrupts
13 * @secondary: pointer to secondary irqaction (force threading)
14 * @thread_flags:  flags related to @thread
15 * @thread_mask:  bitmask for keeping track of @thread activity
16 * @dir:       pointer to the proc/irq/NN/name entry
17 */
18 struct irqaction {
19     irq_handler_t      handler;
20     union {
21         void           *dev_id;
22         void __percpu  *percpu_dev_id;
23     };
24     const struct cpumask *affinity;
25     struct irqaction    *next;
26     irq_handler_t      thread_fn;
27     struct task_struct  *thread;
28     struct irqaction    *secondary;
29     unsigned int        irq;
30     unsigned int        flags;
31     unsigned long       thread_flags;
32     unsigned long       thread_mask;
33     const char          *name;
34     struct proc_dir_entry *dir;
35 }

```

Листинг 4. struct irqaction

enum irqreturn

```

1  /**
2  * enum irqreturn - irqreturn type values
3  * @IRQ_NONE:      interrupt was not from this device or was not handled
4  * @IRQ_HANDLED:   interrupt was handled by this device
5  * @IRQ_WAKE_THREAD: handler requests to wake the handler thread
6  */
7  enum irqreturn {
8      IRQ_NONE      = (0 << 0),
9      IRQ_HANDLED   = (1 << 0),
10     IRQ_WAKE_THREAD = (1 << 1),
11 };

```

Листинг 5. enum irqreturn

softirq

```

1  /* PLEASE, avoid to allocate new softirqs, if you need not _really_ high
2  frequency threaded job scheduling. For almost all the purposes

```

```

3   tasklets are more than enough. F.e. all serial device BHs et
4   al. should be converted to tasklets, not to softirqs.
5   */
6
7   enum
8   {
9       HI_SOFTIRQ=0,
10      TIMER_SOFTIRQ,
11      NET_TX_SOFTIRQ,
12      NET_RX_SOFTIRQ,
13      BLOCK_SOFTIRQ,
14      IRQ_POLL_SOFTIRQ,
15      TASKLET_SOFTIRQ,
16      SCHED_SOFTIRQ,
17      HRTIMER_SOFTIRQ,
18      RCU_SOFTIRQ, /* Preferable RCU should always be the last softirq */
19
20      NR_SOFTIRQS
21  };

```

Листинг 6. softirq

tasklet

```

1  /* Tasklets --- multithreaded analogue of BHs.
2
3   This API is deprecated. Please consider using threaded IRQs instead:
4   https://lore.kernel.org/lkml/20200716081538.2sivhkj4hcyrusem@linutronix
5   .de
6
7   Main feature differing them of generic softirqs: tasklet
8   is running only on one CPU simultaneously.
9
10  Main feature differing them of BHs: different tasklets
11  may be run simultaneously on different CPUs.
12
13  Properties:
14  * If tasklet_schedule() is called, then tasklet is guaranteed
15    to be executed on some cpu at least once after this.
16  * If the tasklet is already scheduled, but its execution is still not
17    started, it will be executed only once.
18  * If this tasklet is already running on another CPU (or schedule is
19    called from tasklet itself), it is rescheduled for later.
20  * Tasklet is strictly serialized wrt itself, but not
21    wrt another tasklets. If client needs some intertask synchronization,
22    he makes it with spinlocks.
23  */
24  struct tasklet_struct
25  {
26      struct tasklet_struct *next;
27      unsigned long state;
28      atomic_t count;
29      bool use_callback;
30      union {
31          void (*func)(unsigned long data);
32          void (*callback)(struct tasklet_struct *t);
33      };

```

```

34 |     unsigned long data;
35 | };
36 |
37 | #define DECLARE_TASKLET(name, _callback) \
38 | struct tasklet_struct name = { \
39 |     .count = ATOMIC_INIT(0), \
40 |     .callback = _callback, \
41 |     .use_callback = true, \
42 | }
43 |
44 | #define DECLARE_TASKLET_DISABLED(name, _callback) \
45 | struct tasklet_struct name = { \
46 |     .count = ATOMIC_INIT(1), \
47 |     .callback = _callback, \
48 |     .use_callback = true, \
49 | }
50 |
51 | #define from_tasklet(var, callback_tasklet, tasklet_fieldname) \
52 |     container_of(callback_tasklet, typeof(*var), tasklet_fieldname)
53 |
54 | #define DECLARE_TASKLET_OLD(name, _func) \
55 | struct tasklet_struct name = { \
56 |     .count = ATOMIC_INIT(0), \
57 |     .func = _func, \
58 | }
59 |
60 | #define DECLARE_TASKLET_DISABLED_OLD(name, _func) \
61 | struct tasklet_struct name = { \
62 |     .count = ATOMIC_INIT(1), \
63 |     .func = _func, \
64 | }
65 |
66 | enum
67 | {
68 |     TASKLET_STATE_SCHED, /* Tasklet is scheduled for execution */
69 |     TASKLET_STATE_RUN /* Tasklet is running (SMP only) */
70 | };
71 |
72 |
73 | void tasklet_init(struct tasklet_struct *t,
74 |                 void (*func)(unsigned long), unsigned long data)
75 | {
76 |     t->next = NULL;
77 |     t->state = 0;
78 |     atomic_set(&t->count, 0);
79 |     t->func = func;
80 |     t->use_callback = false;
81 |     t->data = data;
82 | }
83 |
84 |
85 | static void __tasklet_schedule_common(struct tasklet_struct *t,
86 |                                     struct tasklet_head __percpu *headp,
87 |                                     unsigned int softirq_nr)
88 | {
89 |     struct tasklet_head *head;
90 |     unsigned long flags;
91 |
92 |     local_irq_save(flags);
93 |     head = this_cpu_ptr(headp);
94 |     t->next = NULL;

```

```

95     *head->tail = t;
96     head->tail = &(t->next);
97     raise_softirq_irqoff(softirq_nr);
98     local_irq_restore(flags);
99 }
100
101 void __tasklet_schedule(struct tasklet_struct *t)
102 {
103     __tasklet_schedule_common(t, &tasklet_vec,
104                               TASKLET_SOFTIRQ);
105 }
106 EXPORT_SYMBOL(__tasklet_schedule);
107
108 void __tasklet_hi_schedule(struct tasklet_struct *t)
109 {
110     __tasklet_schedule_common(t, &tasklet_hi_vec,
111                               HI_SOFTIRQ);
112 }
113 EXPORT_SYMBOL(__tasklet_hi_schedule);

```

Листинг 7. tasklet

struct worker

```

1 struct worker_pool;
2
3 /*
4  * The poor guys doing the actual heavy lifting. All on-duty workers are
5  * either serving the manager role, on idle list or on busy hash. For
6  * details on the locking annotation (L, I, X...), refer to workqueue.c.
7  *
8  * Only to be used in workqueue and async.
9  */
10 struct worker {
11     /* on idle list while idle, on busy hash table while busy */
12     union {
13         struct list_head entry; /* L: while idle */
14         struct hlist_node hentry; /* L: while busy */
15     };
16
17     struct work_struct *current_work; /* K: work being processed and its
18     */
19     work_func_t current_func; /* K: function */
20     struct pool_workqueue *current_pwq; /* K: pwq */
21     u64 current_at; /* K: runtime at start or last wakeup */
22     unsigned long current_start; /* K: start time of current work
23     item */
24     unsigned int current_color; /* K: color */
25
26     int sleeping; /* S: is worker sleeping? */
27
28     /* used by the scheduler to determine a worker's last known identity
29     */
30     work_func_t last_func; /* K: last work's fn */
31
32     struct list_head scheduled; /* L: scheduled works */
33
34     struct task_struct *task; /* I: worker task */

```

```

32 struct worker_pool *pool; /* A: the associated pool */
33 /* L: for rescuers */
34 struct list_head node; /* A: anchored at pool->workers */
35 /* A: runs through worker->node */
36
37 unsigned long last_active; /* K: last active timestamp */
38 unsigned int flags; /* L: flags */
39 int id; /* I: worker id */
40
41 /*
42 * Opaque string set with work_set_desc(). Printed out with task
43 * dump for debugging - WARN, BUG, panic or sysrq.
44 */
45 char desc[WORKER_DESC_LEN];
46
47 /* used only by rescuers to point to the target workqueue */
48 struct workqueue_struct *rescue_wq; /* I: the workqueue to rescue */
49 };

```

ЛИСТИНГ 8. struct worker

struct worker_pool

```

1
2 struct worker_pool {
3     raw_spinlock_t lock; /* the pool lock */
4     int cpu; /* I: the associated cpu */
5     int node; /* I: the associated node ID */
6     int id; /* I: pool ID */
7     unsigned int flags; /* L: flags */
8
9     unsigned long last_progress_ts; /* L: last forward progress
10     timestamp */
11     bool cpu_stall; /* WD: stalled cpu bound pool */
12
13     /*
14     * The counter is incremented in a process context on the associated
15     * CPU
16     * w/ preemption disabled, and decremented or reset in the same
17     * context
18     * but w/ pool->lock held. The readers grab pool->lock and are
19     * guaranteed to see if the counter reached zero.
20     */
21     int nr_running;
22
23     struct list_head worklist; /* L: list of pending works */
24
25     int nr_workers; /* L: total number of workers */
26     int nr_idle; /* L: currently idle workers */
27
28     struct list_head idle_list; /* L: list of idle workers */
29     struct timer_list idle_timer; /* L: worker idle timeout */
30     struct work_struct idle_cull_work; /* L: worker idle cleanup */
31
32     struct timer_list mayday_timer; /* L: SOS timer for workers */
33
34     /* a workers is either on busy_hash or idle_list, or the manager */
35     DECLARE_HASHTABLE(busy_hash, BUSY_WORKER_HASH_ORDER);

```

```

33                                     /* L: hash of busy workers */
34
35     struct worker      *manager;    /* L: purely informational */
36     struct list_head   workers;     /* A: attached workers */
37
38     struct ida          worker_ida; /* worker IDs for task name */
39
40     struct workqueue_attrs *attrs;   /* I: worker attributes */
41     struct hlist_node   hash_node;  /* PL: unbound_pool_hash node */
42     int                 refcnt;      /* PL: refcnt for unbound pools */
43 #ifdef CONFIG_PREEMPT_RT
44     spinlock_t          cb_lock;     /* BH worker cancel lock */
45 #endif
46     /*
47      * Destruction of pool is RCU protected to allow dereferences
48      * from get_work_pool().
49      */
50     struct rcu_head      rcu;
51 };

```

Листинг 9. struct worker_pool

Инициализация очереди работ

```

1  #define DECLARE_WORK(n, f) \
2      struct work_struct n = __WORK_INITIALIZER(n, f)
3
4  #define __INIT_WORK(_work, _func, _onstack) \
5      do { \
6          static __maybe_unused struct lock_class_key __key; \
7          __INIT_WORK_KEY(_work, _func, _onstack, &__key); \
8      } while (0)
9
10
11 #define INIT_WORK(_work, _func) \
12     __INIT_WORK((_work), (_func), 0)
13
14 int queue_work(struct workqueue_struct *wq, struct work_struct *work);
15 int queue_work_on(int cpu, struct workqueue_struct *wq, struct work_struct *work);
16 int queue_delayed_work(struct workqueue_struct *wq, struct work_struct *work, unsigned long delay);

```

Листинг 10. Инициализация очереди работ

struct socket

```

1  /**
2   *
3   * enum sock_type - Socket types
4   * @SOCK_STREAM: stream (connection) socket
5   * @SOCK_DGRAM: datagram (conn. less) socket
6   * @SOCK_RAW: raw socket
7   * @SOCK_RDM: reliably-delivered message
8   * @SOCK_SEQPACKET: sequential packet socket

```

```

9  * @SOCK_DCCP: Datagram Congestion Control Protocol socket
10 * @SOCK_PACKET: linux specific way of getting packets at the dev level.
11 *      For writing rarp and other similar things on the user level.
12 *
13 * When adding some new socket type please
14 * grep ARCH_HAS_SOCKET_TYPE include/asm-* /socket.h, at least MIPS
15 * overrides this enum for binary compat reasons.
16 */
17 enum sock_type {
18     SOCK_STREAM = 1,
19     SOCK_DGRAM  = 2,
20     SOCK_RAW    = 3,
21     SOCK_RDM    = 4,
22     SOCK_SEQPACKET = 5,
23     SOCK_DCCP   = 6,
24     SOCK_PACKET = 10,
25 };
26
27
28 struct socket {
29     socket_state      state;
30
31     short             type;
32
33     unsigned long     flags;
34
35     struct file       *file;
36     struct sock       *sk;
37     const struct proto_ops *ops; /* Might change with IPV6_ADDRFORM or
38     MPTCP. */
39
40     struct socket_wq   wq;
41 };

```

Листинг 11. struct socket

Сокеты AF_INET, сетевой стек

```

1  struct in_addr {
2      __be32 s_addr;
3  };
4
5  /* Structure describing an Internet (IP) socket address. */
6  #if __UAPI_DEF_SOCKADDR_IN
7  #define __SOCK_SIZE__ 16 /* sizeof(struct sockaddr) */
8  struct sockaddr_in {
9      __kernel_sa_family_t sin_family; /* Address family */
10     __be16 sin_port; /* Port number */
11     struct in_addr sin_addr; /* Internet address */
12
13     /* Pad to size of 'struct sockaddr'. */
14     unsigned char __pad[__SOCK_SIZE__ - sizeof(short int) -
15         sizeof(unsigned short int) - sizeof(struct in_addr)];
16 };

```

Листинг 12. Сокеты AF_INET, сетевой стек

sys_socketcall

```
1 asmlinkage long sys_socketcall(int call, unsigned long *args)
2 {
3     int err;
4     if (copy_from_user(a, args, args[call]))
5         return -EFAULT;
6
7     a0 = a[0];
8     a1 = a[1];
9
10    switch (call)
11    {
12        case SYS_SOCKET:
13            err = sys_socket(a0, a1, a[2]);
14            break;
15        case SYS_BIND:
16            err = sys_bind(a0, (struct sockaddr *) a1, a[2]);
17            break;
18        case SYS_CONNECT:
19            err = sys_connect(a0, (struct sockaddr *) a1, a[2]);
20            break;
21        ...
22    }
23
24    return err;
25 }
```

Листинг 13. sys_socketcall

sys_socket

```
1 asmlinkage long sys_socket(int family, int type, int protocol)
2 {
3     int retval;
4     struct socket *sock;
5     ...
6     retval = socket_create(family, type, protocol, &sock);
7     ...
8     return retval;
9 }
```

Листинг 14. sys_socket

struct proc_dir_entry

```
1 struct proc_dir_entry {
2     /*
3     * number of callers into module in progress;
4     * negative -> it's going away RSN
5     */
6     atomic_t in_use;
7     refcount_t refcnt;
8     struct list_head pde_openers; /* who did ->open, but not ->release
9     */
10 }
```

```

9      /* protects ->pde_openers and all struct pde_opener instances */
10     spinlock_t pde_unload_lock;
11     struct completion *pde_unload_completion;
12     const struct inode_operations *proc_iops;
13     union {
14         const struct proc_ops *proc_ops;
15         const struct file_operations *proc_dir_ops;
16     };
17     union {
18         const struct seq_operations *seq_ops;
19         int (*single_show)(struct seq_file *, void *);
20     };
21     proc_write_t write;
22     void *data;
23     unsigned int state_size;
24     unsigned int low_ino;
25     nlink_t nlink;
26     kuid_t uid;
27     kgid_t gid;
28     loff_t size;
29     struct proc_dir_entry *parent;
30     struct rb_root subdir;
31     struct rb_node subdir_node;
32     char *name;
33     umode_t mode;
34     u8 flags;
35     u8 namelen;
36     char inline_name[];
37 }

```

Листинг 15. struct proc_dir_entry

struct proc_ops

```

1 extern struct proc_dir_entry *proc_create_data(const char *, umode_t,
2                                               struct proc_dir_entry *,
3                                               const struct proc_ops *,
4                                               void *);
5
6 struct proc_dir_entry *proc_create(const char *name, umode_t mode, struct
7 proc_dir_entry *parent, const struct proc_ops *proc_ops);
8
9 extern struct proc_dir_entry *proc_mkdir(const char *, struct
10 proc_dir_entry *);
11
12 extern struct proc_dir_entry *proc_symlink(const char *,
13 struct proc_dir_entry *, const char *);
14
15 extern void remove_proc_entry(const char *, struct proc_dir_entry *);
16
17 struct proc_ops {
18     unsigned int proc_flags;
19     int (*proc_open)(struct inode *, struct file *);
20     ssize_t (*proc_read)(struct file *, char __user *, size_t, loff_t *);
21     ssize_t (*proc_read_iter)(struct kiocb *, struct iov_iter *);
22     ssize_t (*proc_write)(struct file *, const char __user *, size_t,
23 loff_t *);
24     /* mandatory unless nonseekable_open() or equivalent is used */

```

```

23     loff_t  (*proc_lseek)(struct file *, loff_t, int);
24     int  (*proc_release)(struct inode *, struct file *);
25     __poll_t  (*proc_poll)(struct file *, struct poll_table_struct *);
26     long  (*proc_ioctl)(struct file *, unsigned int, unsigned long);
27 #ifdef CONFIG_COMPAT
28     long  (*proc_compat_ioctl)(struct file *, unsigned int, unsigned
29         long);
29 #endif
30     int  (*proc_mmap)(struct file *, struct vm_area_struct *);
31     unsigned long  (*proc_get_unmapped_area)(struct file *, unsigned long,
32         unsigned long, unsigned long, unsigned long);
32 } __randomize_layout;
33
34 static const struct proc_ops device_proc_ops = {
35     .proc_open = device_open,
36     .proc_read = device_read,
37     .proc_write = device_write,
38     .proc_release = device_release,
39 };

```

Листинг 16. struct proc_ops

file_system_type

```

1 struct file_system_type {
2     const char *name;
3     int fs_flags;
4 #define FS_REQUIRES_DEV 1
5 #define FS_BINARY_MOUNTDATA 2
6 #define FS_HAS_SUBTYPE 4
7 #define FS_USERNS_MOUNT 8 /* Can be mounted by userns root */
8 #define FS_DISALLOW_NOTIFY_PERM 16 /* Disable fanotify permission events
9     */
9 #define FS_ALLOW_IDMAP 32 /* FS has been updated to handle
10     vfs idmappings. */
10 #define FS_MGTIME 64 /* FS uses multigrain timestamps */
11 #define FS_LBS 128 /* FS supports LBS */
12 #define FS_POWER_FREEZE 256 /* Always freeze on suspend/hibernate */
13 #define FS_RENAME_DOES_D_MOVE 32768 /* FS will handle d_move() during
14     rename() internally. */
14     int (*init_fs_context)(struct fs_context *);
15     const struct fs_parameter_spec *parameters;
16     void (*kill_sb)(struct super_block *);
17     struct module *owner;
18     struct file_system_type *next;
19     struct hlist_head fs_supers;
20
21     struct lock_class_key s_lock_key;
22     struct lock_class_key s_umount_key;
23     struct lock_class_key s_vfs_rename_key;
24     struct lock_class_key s_writers_key[SB_FREEZE_LEVELS];
25
26     struct lock_class_key i_lock_key;
27     struct lock_class_key i_mutex_key;
28     struct lock_class_key invalidate_lock_key;
29     struct lock_class_key i_mutex_dir_key;
30 };

```

Листинг 17. file_system_type

struct super_block

```

1 struct super_block {
2     struct list_head      s_list;      /* Keep this first */
3     dev_t                  s_dev;        /* search index; _not_ kdev_t */
4     unsigned char          s_blocksize_bits;
5     unsigned long          s_blocksize;
6     loff_t                 s_maxbytes; /* Max file size */
7     struct file_system_type *s_type;
8     const struct super_operations *s_op;
9     const struct dquot_operations *dq_op;
10    const struct quotactl_ops *s_qcop;
11    const struct export_operations *s_export_op;
12    unsigned long           s_flags;
13    unsigned long           s_iflags; /* internal SB_I_* flags */
14    unsigned long           s_magic;
15    struct dentry            *s_root;
16    struct rw_semaphore     s_umount;
17    int                     s_count;
18    atomic_t                s_active;
19 #ifdef CONFIG_SECURITY
20     void                   *s_security;
21 #endif
22     const struct xattr_handler *const *s_xattr;
23 #ifdef CONFIG_FS_ENCRYPTION
24     const struct fscrypt_operations *s_cop;
25     struct fscrypt_keyring *s_master_keys; /* master crypto keys
        in use */
26 #endif
27 #ifdef CONFIG_FS_VERITY
28     const struct fsverity_operations *s_vop;
29 #endif
30 #if IS_ENABLED(CONFIG_UNICODE)
31     struct unicode_map      *s_encoding;
32     ul6                     s_encoding_flags;
33 #endif
34     struct hlist_bl_head    s_roots; /* alternate root dentries
        for NFS */
35     struct mount            *s_mounts; /* list of mounts; _not_ for
        fs use */
36     struct block_device     *s_bdev; /* can go away once we use an
        accessor for @s_bdev_file */
37     struct file             *s_bdev_file;
38     struct backing_dev_info *s_bdi;
39     struct mtd_info         *s_mtd;
40     struct hlist_node       s_instances;
41     unsigned int            s_quota_types; /* Bitmask of supported
        quota types */
42     struct quota_info       s_dquot; /* Diskquota specific options
        */
43
44     struct sb_writers       s_writers;
45
46     /*
47     * Keep s_fs_info, s_time_gran, s_fsnotify_mask, and
48     * s_fsnotify_info together for cache efficiency. They are frequently
49     * accessed and rarely modified.
50     */

```

```

51     void                                *s_fs_info; /* Filesystem private info */
52
53     /* Granularity of c/m/ptime in ns (cannot be worse than a second) */
54     u32                                s_time_gran;
55     /* Time limits for c/m/ptime in seconds */
56     time64_t                           s_time_min;
57     time64_t                           s_time_max;
58 #ifdef CONFIG_FSNOTIFY
59     u32                                s_fsnotify_mask;
60     struct fsnotify_sb_info            *s_fsnotify_info;
61 #endif
62
63     char                                s_id[32]; /* Informational name */
64     uuid_t                              s_uuid; /* UUID */
65     u8                                  s_uuid_len; /* Default 16, possibly smaller for
66         weird filesystems */
67
68     /* if set, fs shows up under sysfs at /sys/fs/$FSTYP/s_sysfs_name */
69     char                                s_sysfs_name[UUID_STRING_LEN + 1];
70
71     unsigned int                        s_max_links;
72     unsigned int                        s_d_flags; /* default d_flags for
73         dentries */
74
75     /*
76      * The next field is for VFS *only*. No filesystems have any business
77      * even looking at it. You had been warned.
78      */
79     struct mutex                        s_vfs_rename_mutex; /* Kludge */
80
81     /*
82      * Filesystem subtype. If non-empty the filesystem type field
83      * in /proc/mounts will be "type.subtype"
84      */
85     const char                          *s_subtype;
86
87     const struct dentry_operations      *__s_d_op; /* default d_op for
88         dentries */
89
90     struct shrinker                     *s_shrink; /* per-sb shrinker handle */
91
92     /* Number of inodes with nlink == 0 but still referenced */
93     atomic_long_t                       s_remove_count;
94
95     /* Read-only state of the superblock is being changed */
96     int                                  s_readonly_remount;
97
98     /* per-sb errseq_t for reporting writeback errors via syncfs */
99     errseq_t s_wb_err;
100
101     /* AIO completions deferred from interrupt context */
102     struct workqueue_struct             *s_dio_done_wq;
103     struct hlist_head                   s_pins;
104
105     /*
106      * Owning user namespace and default context in which to
107      * interpret filesystem uids, gids, quotas, device nodes,
108      * xattrs and security labels.
109      */
110     struct user_namespace               *s_user_ns;

```

```

109 | /*
110 |  * The list_lru structure is essentially just a pointer to a table
111 |  * of per-node lru lists, each of which has its own spinlock.
112 |  * There is no need to put them into separate cachelines.
113 |  */
114 | struct list_lru          s_dentry_lru;
115 | struct list_lru          s_inode_lru;
116 | struct rcu_head          rcu;
117 | struct work_struct       destroy_work;
118 |
119 | struct mutex             s_sync_lock;    /* sync serialisation lock
120 |      */
121 |
122 | /*
123 |  * Indicates how deep in a filesystem stack this SB is
124 |  */
125 | int s_stack_depth;
126 |
127 | /* s_inode_list_lock protects s_inodes */
128 | spinlock_t              s_inode_list_lock ____cacheline_aligned_in_smp
129 | ;
130 | struct list_head         s_inodes;    /* all inodes */
131 |
132 | spinlock_t              s_inode_wblist_lock;
133 | struct list_head         s_inodes_wb;    /* writeback inodes */
134 | long                    s_min_writeback_pages;
135 |
136 | /* number of fserrors that are being sent to fsnotify/filesystems */
137 | refcount_t              s_pending_errors;
138 | }

```

Листинг 18. struct super_block

struct inode

```

1 struct inode {
2     umode_t          i_mode;
3     unsigned short    i_opflags;
4     unsigned int      i_flags;
5 #ifdef CONFIG_FS_POSIX_ACL
6     struct posix_acl  *i_acl;
7     struct posix_acl  *i_default_acl;
8 #endif
9     kuid_t           i_uid;
10    kgid_t            i_gid;
11
12    const struct inode_operations *i_op;
13    struct super_block *i_sb;
14    struct address_space *i_mapping;
15
16 #ifdef CONFIG_SECURITY
17    void *i_security;
18 #endif
19
20    /* Stat data, not accessed from path walking */
21    unsigned long      i_ino;
22    /*
23     * Filesystems may only read i_nlink directly. They shall use the
24     * following functions for modification:

```

```

25     *
26     *      (set | clear | inc | drop) _nlink
27     *      inode_ (inc | dec) _link_count
28     */
29     union {
30         const unsigned int i_nlink;
31         unsigned int __i_nlink;
32     };
33     dev_t          i_rdev;
34     loff_t         i_size;
35     time64_t       i_atime_sec;
36     time64_t       i_mtime_sec;
37     time64_t       i_ctime_sec;
38     u32            i_atime_nsec;
39     u32            i_mtime_nsec;
40     u32            i_ctime_nsec;
41     u32            i_generation;
42     spinlock_t     i_lock; /* i_blocks, i_bytes, maybe i_size */
43     unsigned short i_bytes;
44     u8             i_blkbits;
45     enum rw_hint   i_write_hint;
46     blkcnt_t       i_blocks;
47
48 #ifdef __NEED_I_SIZE_ORDERED
49     seqcount_t     i_size_seqcount;
50 #endif
51
52     /* Misc */
53     struct inode_state_flags i_state;
54     /* 32-bit hole */
55     struct rw_semaphore i_rwsem;
56
57     unsigned long    dirtied_when; /* jiffies of first dirtying */
58     unsigned long    dirtied_time_when;
59
60     struct hlist_node i_hash;
61     struct list_head i_io_list; /* backing dev IO list */
62 #ifdef CONFIG_CGROUP_WRITEBACK
63     struct bdi_writeback *i_wb; /* the associated cgroup wb */
64
65     /* foreign inode detection, see wbc_detach_inode() */
66     int i_wb_frn_winner;
67     u16 i_wb_frn_avg_time;
68     u16 i_wb_frn_history;
69 #endif
70     struct list_head i_lru; /* inode LRU list */
71     struct list_head i_sb_list;
72     struct list_head i_wb_list; /* backing dev writeback list */
73     union {
74         struct hlist_head i_dentry;
75         struct rcu_head i_rcu;
76     };
77     atomic64_t i_version;
78     atomic64_t i_sequence; /* see futex */
79     atomic_t i_count;
80     atomic_t i_dio_count;
81     atomic_t i_writecount;
82 #if defined(CONFIG_IMA) || defined(CONFIG_FILE_LOCKING)
83     atomic_t i_readcount; /* struct files open RO */
84 #endif
85     union {

```

```

86     const struct file_operations      *i_fop; /* former ->i_op->
        default_file_ops */
87     void (*free_inode)(struct inode *);
88 };
89 struct file_lock_context      *i_flctx;
90 struct address_space          i_data;
91 union {
92     struct list_head          i_devices;
93     int                        i_linklen;
94 };
95 union {
96     struct pipe_inode_info     *i_pipe;
97     struct cdev                *i_cdev;
98     char                       *i_link;
99     unsigned                   i_dir_seq;
100 };
101
102 void                          *i_private; /* fs or device private pointer */
103 }

```

Листинг 19. struct inode

struct inode_operations

```

1 struct inode_operations {
2     struct dentry * (*lookup) (struct inode *, struct dentry *, unsigned
        int);
3     const char * (*get_link) (struct dentry *, struct inode *, struct
        delayed_call *);
4     int (*permission) (struct mnt_idmap *, struct inode *, int);
5     struct posix_acl * (*get_inode_acl)(struct inode *, int, bool);
6
7     int (*readlink) (struct dentry *, char __user *, int);
8
9     int (*create) (struct mnt_idmap *, struct inode *, struct dentry *,
        umode_t, bool);
10    int (*link) (struct dentry *, struct inode *, struct dentry *);
11    int (*unlink) (struct inode *, struct dentry *);
12    int (*symlink) (struct mnt_idmap *, struct inode *, struct dentry *,
        const char *);
13    struct dentry * (*mkdir) (struct mnt_idmap *, struct inode *,
        struct dentry *, umode_t);
14    int (*rmdir) (struct inode *, struct dentry *);
15    int (*mknod) (struct mnt_idmap *, struct inode *, struct dentry *,
        umode_t, dev_t);
16    int (*rename) (struct mnt_idmap *, struct inode *, struct dentry *,
        struct inode *, struct dentry *, unsigned int);
17    int (*setattr) (struct mnt_idmap *, struct dentry *, struct iattr *);
18    int (*getattr) (struct mnt_idmap *, const struct path *,
        struct kstat *, u32, unsigned int);
19    ssize_t (*listxattr) (struct dentry *, char *, size_t);
20    int (*fiemap)(struct inode *, struct fiemap_extent_info *, u64 start,
        u64 len);
21    int (*update_time)(struct inode *inode, enum fs_update_time type,
        unsigned int flags);
22    void (*sync_lazytime)(struct inode *inode);
23    int (*atomic_open)(struct inode *, struct dentry *,
        struct file *, unsigned open_flag,

```

```

33         umode_t create_mode);
34 int (*tmpfile) (struct mnt_idmap *, struct inode *,
35                struct file *, umode_t);
36 struct posix_acl *(*get_acl)(struct mnt_idmap *, struct dentry *,
37                              int);
38 int (*set_acl)(struct mnt_idmap *, struct dentry *,
39               struct posix_acl *, int);
40 int (*fileattr_set)(struct mnt_idmap *idmap,
41                    struct dentry *dentry, struct file_kattr *fa);
42 int (*fileattr_get)(struct dentry *dentry, struct file_kattr *fa);
43 struct offset_ctx *(*get_offset_ctx)(struct inode *inode);
44 }

```

ЛИСТИНГ 20. struct inode_operations

struct dentry

```

1 struct dentry {
2     /* RCU lookup touched fields */
3     unsigned int d_flags; /* protected by d_lock */
4     seqcount_spinlock_t d_seq; /* per dentry seqlock */
5     struct hlist_bl_node d_hash; /* lookup hash list */
6     struct dentry *d_parent; /* parent directory */
7     union {
8         struct qstr __d_name; /* for use ONLY in fs/dcache.c */
9         const struct qstr d_name;
10    };
11    struct inode *d_inode; /* Where the name belongs to - NULL is
12                           * negative */
13    union shortname_store d_shortname;
14    /* --- cacheline 1 boundary (64 bytes) was 32 bytes ago --- */
15
16    /* Ref lookup also touches following */
17    const struct dentry_operations *d_op;
18    struct super_block *d_sb; /* The root of the dentry tree */
19    unsigned long d_time; /* used by d_revalidate */
20    void *d_fsdata; /* fs-specific data */
21    /* --- cacheline 2 boundary (128 bytes) --- */
22    struct lockref d_lockref; /* per-dentry lock and refcount
23                             * keep separate from RCU lookup area if
24                             * possible!
25                             */
26
27    union {
28        struct list_head d_lru; /* LRU list */
29        wait_queue_head_t *d_wait; /* in-lookup ones only */
30    };
31    struct hlist_node d_sib; /* child of parent list */
32    struct hlist_head d_children; /* our children */
33    /*
34     * d_alias and d_rcu can share memory
35     */
36    union {
37        struct hlist_node d_alias; /* inode alias list */
38        struct hlist_bl_node d_in_lookup_hash; /* only for in-lookup ones
39                                                */
40        struct rcu_head d_rcu;
41    } d_u;

```

41 | };

Листинг 21. struct dentry

struct super_operations

```
1 struct super_operations {
2     struct inode *(*alloc_inode)(struct super_block *sb);
3     void (*destroy_inode)(struct inode *inode);
4     void (*free_inode)(struct inode *inode);
5     void (*dirty_inode)(struct inode *inode, int flags);
6     int (*write_inode)(struct inode *inode, struct writeback_control *wbc)
7     ;
8     int (*drop_inode)(struct inode *inode);
9     void (*evict_inode)(struct inode *inode);
10    void (*put_super)(struct super_block *sb);
11    int (*sync_fs)(struct super_block *sb, int wait);
12    int (*freeze_super)(struct super_block *sb, enum freeze_holder who,
13        const void *owner);
14    int (*freeze_fs)(struct super_block *sb);
15    int (*thaw_super)(struct super_block *sb, enum freeze_holder who,
16        const void *owner);
17    int (*unfreeze_fs)(struct super_block *sb);
18    int (*statfs)(struct dentry *dentry, struct kstatfs *kstatfs);
19    void (*umount_begin)(struct super_block *sb);
20
21    int (*show_options)(struct seq_file *seq, struct dentry *dentry);
22    int (*show_devname)(struct seq_file *seq, struct dentry *dentry);
23    int (*show_path)(struct seq_file *seq, struct dentry *dentry);
24    int (*show_stats)(struct seq_file *seq, struct dentry *dentry);
25 #ifdef CONFIG_QUOTA
26     ssize_t (*quota_read)(struct super_block *sb, int type, char *data,
27         size_t len, loff_t off);
28     ssize_t (*quota_write)(struct super_block *sb, int type,
29         const char *data, size_t len, loff_t off);
30     struct dquot __rcu **(*get_dquots)(struct inode *inode);
31 #endif
32     long (*nr_cached_objects)(struct super_block *sb,
33         struct shrink_control *sc);
34     long (*free_cached_objects)(struct super_block *sb,
35         struct shrink_control *sc);
36
37     /*
38      * If a filesystem can support graceful removal of a device and
39      * continue read-write operations, implement this callback.
40      *
41      * Return 0 if the filesystem can continue read-write.
42      * Non-zero return value or no such callback means the fs will be
43      * shutdown
44      * as usual.
45      */
46     int (*remove_bdev)(struct super_block *sb, struct block_device *bdev);
47     void (*shutdown)(struct super_block *sb);
48
49     /* Report a filesystem error */
50     void (*report_error)(const struct fserror_event *event);
51 };
52
```

Листинг 22. struct super_operations

struct FILE

```
1 typedef struct _IO_FILE FILE;
```

Листинг 23. struct FILE

struct _IO_FILE

```
1 struct _IO_FILE {
2     int _flags; /* High-order word is _IO_MAGIC; rest is flags. */
3 #define _IO_file_flags _flags
4
5     /* The following pointers correspond to the C++ streambuf protocol. */
6     /* Note: Tk uses the _IO_read_ptr and _IO_read_end fields directly. */
7     char* _IO_read_ptr; /* Current read pointer */
8     char* _IO_read_end; /* End of get area. */
9     char* _IO_read_base; /* Start of putback+get area. */
10    char* _IO_write_base; /* Start of put area. */
11    char* _IO_write_ptr; /* Current put pointer. */
12    char* _IO_write_end; /* End of put area. */
13    char* _IO_buf_base; /* Start of reserve area. */
14    char* _IO_buf_end; /* End of reserve area. */
15    /* The following fields are used to support backing up and undo. */
16    char* _IO_save_base; /* Pointer to start of non-current get area. */
17    char* _IO_backup_base; /* Pointer to first valid character of backup
18        area */
19    char* _IO_save_end; /* Pointer to end of non-current get area. */
20
21    struct _IO_marker* _markers;
22
23    struct _IO_FILE* _chain;
24
25    int _fileno;
26 #if 0
27     int _blksize;
28 #else
29     int _flags2;
30 #endif
31     _IO_off_t _old_offset; /* This used to be _offset but it's too small.
32        */
33
34 #define __HAVE_COLUMN /* temporary */
35     /* 1+column number of pbase(); 0 is unknown. */
36     unsigned short _cur_column;
37     signed char _vtable_offset;
38     char _shortbuf[1];
39
40     /* char* _save_gptr; char* _save_egptr; */
41
42     _IO_lock_t* _lock;
43 #ifndef _IO_USE_OLD_IO_FILE
44 };
45 #endif
```

Листинг 24. struct _IO_FILE

struct stat

```
1 struct stat {
2     dev_t          st_dev;          /* [XSI] ID of device containing file
3     /*
4     ino_t          st_ino;          /* [XSI] File serial number */
5     mode_t         st_mode;        /* [XSI] Mode of file (see below) */
6     nlink_t        st_nlink;       /* [XSI] Number of hard links */
7     uid_t          st_uid;         /* [XSI] User ID of the file */
8     gid_t          st_gid;         /* [XSI] Group ID of the file */
9     dev_t          st_rdev;        /* [XSI] Device ID */
10    #if !defined(_POSIX_C_SOURCE) || defined(_DARWIN_C_SOURCE)
11    struct timespec st_atimespec;    /* time of last access */
12    struct timespec st_mtimespec;    /* time of last data modification */
13    struct timespec st_ctimespec;    /* time of last status change */
14    #else
15    time_t          st_atime;        /* [XSI] Time of last access */
16    long            st_atimensec;    /* nsec of last access */
17    time_t          st_mtime;        /* [XSI] Last data modification time
18    /*
19    long            st_mtimensec;     /* last data modification nsec */
20    time_t          st_ctime;        /* [XSI] Time of last status change */
21    long            st_ctimensec;    /* nsec of last status change */
22    #endif
23    off_t           st_size;         /* [XSI] file size, in bytes */
24    blkcnt_t        st_blocks;       /* [XSI] blocks allocated for file */
25    blksize_t       st_blksize;      /* [XSI] optimal blocksize for I/O */
26    __uint32_t      st_flags;        /* user defined flags for file */
27    __uint32_t      st_gen;          /* file generation number */
28    __int32_t       st_lspare;       /* RESERVED: DO NOT USE! */
29    __int64_t       st_qspare[2];    /* RESERVED: DO NOT USE! */
30 };
```

Листинг 25. struct stat

struct fs_struct

```
1 struct fs_struct {
2     int users;
3     seqlock_t seq;
4     int umask;
5     int in_exec;
6     struct path root, pwd;
7 }
```

Листинг 26. struct fs_struct

struct file

```
1 /**
2  * struct file - Represents a file
3  * @f_lock: Protects f_ep, f_flags. Must not be taken from IRQ context.
4  * @f_mode: FMODE_* flags often used in hotpaths
5  * @f_op: file operations
6  * @f_mapping: Contents of a cacheable, mappable object.
```

```

7  * @private_data: filesystem or driver specific data
8  * @f_inode: cached inode
9  * @f_flags: file flags
10 * @f_iocb_flags: iocb flags
11 * @f_cred: stashed credentials of creator/opener
12 * @f_owner: file owner
13 * @f_path: path of the file
14 * @__f_path: writable alias for @f_path; *ONLY* for core VFS and only
    before
15 *     the file gets open
16 * @f_pos_lock: lock protecting file position
17 * @f_pipe: specific to pipes
18 * @f_pos: file position
19 * @f_security: LSM security context of this file
20 * @f_wb_err: writeback error
21 * @f_sb_err: per sb writeback errors
22 * @f_ep: link of all epoll hooks for this file
23 * @f_task_work: task work entry point
24 * @f_llist: work queue entrypoint
25 * @f_ra: file's readahead state
26 * @f_freeptr: Pointer used by SLAB_TYPESAFE_BY_RCU file cache (don't
    touch.)
27 * @f_ref: reference count
28 */
29 struct file {
30     spinlock_t          f_lock;
31     fmode_t             f_mode;
32     const struct file_operations *f_op;
33     struct address_space *f_mapping;
34     void                *private_data;
35     struct inode         *f_inode;
36     unsigned int         f_flags;
37     unsigned int         f_iocb_flags;
38     const struct cred     *f_cred;
39     struct fown_struct   *f_owner;
40     /* --- cacheline 1 boundary (64 bytes) --- */
41     union {
42         const struct path f_path;
43         struct path      __f_path;
44     };
45     union {
46         /* regular files (with FMODE_ATOMIC_POS) and directories */
47         struct mutex      f_pos_lock;
48         /* pipes */
49         u64               f_pipe;
50     };
51     loff_t               f_pos;
52 #ifdef CONFIG_SECURITY
53     void                 *f_security;
54 #endif
55     /* --- cacheline 2 boundary (128 bytes) --- */
56     errseq_t             f_wb_err;
57     errseq_t             f_sb_err;
58 #ifdef CONFIG_EPOLL
59     struct hlist_head    *f_ep;
60 #endif
61     union {
62         struct callback_head f_task_work;
63         struct llist_node    f_llist;
64         struct file_ra_state f_ra;
65         freeptr_t            f_freeptr;

```

```

66     };
67     file_ref_t      f_ref;
68     /* --- cacheline 3 boundary (192 bytes) --- */
69 }

```

Листинг 27. struct file

struct vfsmount

```

1 struct vfsmount {
2     struct dentry *mnt_root; /* root of the mounted tree */
3     struct super_block *mnt_sb; /* pointer to superblock */
4     int mnt_flags;
5     struct mnt_idmap *mnt_idmap;
6 }

```

Листинг 28. struct vfsmount

struct file_operations

```

1 struct file_operations {
2     struct module *owner;
3     fop_flags_t fop_flags;
4     loff_t (*llseek) (struct file *, loff_t, int);
5     ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
6     ssize_t (*write) (struct file *, const char __user *, size_t, loff_t
7     *);
8     ssize_t (*read_iter) (struct kiocb *, struct iov_iter *);
9     ssize_t (*write_iter) (struct kiocb *, struct iov_iter *);
10    int (*iopoll)(struct kiocb *kiocb, struct io_comp_batch *,
11        unsigned int flags);
12    int (*iterate_shared) (struct file *, struct dir_context *);
13    __poll_t (*poll) (struct file *, struct poll_table_struct *);
14    long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
15    long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
16    int (*mmap) (struct file *, struct vm_area_struct *);
17    int (*open) (struct inode *, struct file *);
18    int (*flush) (struct file *, fl_owner_t id);
19    int (*release) (struct inode *, struct file *);
20    int (*fsync) (struct file *, loff_t, loff_t, int datasync);
21    int (*fasync) (int, struct file *, int);
22    int (*lock) (struct file *, int, struct file_lock *);
23    unsigned long (*get_unmapped_area)(struct file *, unsigned long,
24        unsigned long, unsigned long, unsigned long);
25    int (*check_flags)(int);
26    int (*flock) (struct file *, int, struct file_lock *);
27    ssize_t (*splice_write)(struct pipe_inode_info *, struct file *,
28        loff_t *, size_t, unsigned int);
29    ssize_t (*splice_read)(struct file *, loff_t *, struct pipe_inode_info
30        *, size_t, unsigned int);
31    void (*splice_eof)(struct file *file);
32    int (*setlease)(struct file *, int, struct file_lease **, void **);
33    long (*fallocate)(struct file *file, int mode, loff_t offset,
34        loff_t len);
35    void (*show_fdinfo)(struct seq_file *m, struct file *f);

```

```

32 #ifndef CONFIG_MMU
33     unsigned (*mmap_capabilities)(struct file *);
34 #endif
35     ssize_t (*copy_file_range)(struct file *, loff_t, struct file *,
36                               loff_t, size_t, unsigned int);
37     loff_t (*remap_file_range)(struct file *file_in, loff_t pos_in,
38                               struct file *file_out, loff_t pos_out,
39                               loff_t len, unsigned int remap_flags);
40     int (*fadvise)(struct file *, loff_t, loff_t, int);
41     int (*uring_cmd)(struct io_uring_cmd *ioucmd, unsigned int issue_flags
42                      );
43     int (*uring_cmd_iopoll)(struct io_uring_cmd *, struct io_comp_batch *,
44                             unsigned int poll_flags);
45     int (*mmap_prepare)(struct vm_area_desc *);
46 }

```

Листинг 29. struct file_operations

Загрузка модуля

```

1  #include <linux/module.h>
2  #include <linux/init.h>
3
4  static int __init md_init(void)
5  {
6      printk("Module_loaded!\n");
7      return 0;
8  }
9
10
11 static int __init md_init(void)
12 {
13     struct task_struct *task = &init_task;
14     do {
15         printk(KERN_INFO "Task: _pid: %d, _comm: %s, _ppid: %d, _state: %ld, _
16                flags: %x, _prio: %d, _policy: %d, _root: %s, _pwd: %s\n",
17                task->pid, task->comm, task->parent->pid,
18                task->state, task->flags, task->prio, task->policy,
19                task->fs->root.dentry->d_iname, task->fs->pwd.dentry->d_name.
20                name);
21     } while ((task = next_task(task)) != &init_task);
22     printk(KERN_INFO "Current: _pid: %d, _comm: %s, _ppid: %d, _state: %ld, _
23            flags: %x, _prio: %d, _policy: %d, _root: %s, _pwd: %s\n",
24            current->pid, current->comm, current->parent->pid,
25            current->state, current->flags, current->prio, current->policy,
26            current->fs->root.dentry->d_iname, current->fs->pwd.dentry->d_name
27            .name);
28     return 0;
29 }
30
31 static void __exit md_exit(void)
32 {
33     printk("Module_unloaded\n");
34 }
35
36 module_init(md_init);
37 module_exit(md_exit);
38 }

```

Листинг 30. Загрузка модуля

printk flags

```
1 #define KERN_EMERG KERN_SOH "0" /* system is unusable */
2 #define KERN_ALERT KERN_SOH "1" /* action must be taken immediately */
3 #define KERN_CRIT KERN_SOH "2" /* critical conditions */
4 #define KERN_ERR KERN_SOH "3" /* error conditions */
5 #define KERN_WARNING KERN_SOH "4" /* warning conditions */
6 #define KERN_NOTICE KERN_SOH "5" /* normal but significant condition */
7 #define KERN_INFO KERN_SOH "6" /* informational */
8 #define KERN_DEBUG KERN_SOH "7" /* debug-level messages */
9 }
```

Листинг 31. printk flags

Функции для работы с кэшем SLAB

```
1 struct kmem_cache *kmem_cache_create(const char *name, size_t size,
   size_t offset,
2                                     unsigned long flags, void (*
   constructor)(void *));
3
4 void kmem_cache_destroy(struct kmem_cache *cache);
5
6 void *kmem_cache_alloc(struct kmem_cache *cache, gfp_t flags);
7
8 void kmem_cache_free(struct kmem_cache *cache, void *obj);
9 }
```

Листинг 32. Функции для работы с кэшем SLAB

simple_fill_super

```
1 int simple_fill_super(struct super_block *s, unsigned long magic,
2                       const struct tree_descr *files)
3 {
4     struct inode *inode;
5     struct dentry *dentry;
6     int i;
7
8     s->s_blocksize = PAGE_SIZE;
9     s->s_blocksize_bits = PAGE_SHIFT;
10    s->s_magic = magic;
11    s->s_op = &simple_super_operations;
12    s->s_time_gran = 1;
13
14    inode = new_inode(s);
15    if (!inode)
16        return -ENOMEM;
17    /*
18     * because the root inode is 1, the files array must not contain an
19     * entry at index 1
20     */
21 }
```

```

21 | inode->i_ino = 1;
22 | inode->i_mode = S_IFDIR | 0755;
23 | simple_inode_init_ts(inode);
24 | inode->i_op = &simple_dir_inode_operations;
25 | inode->i_fop = &simple_dir_operations;
26 | set_nlink(inode, 2);
27 | s->s_root = d_make_root(inode);
28 | if (!s->s_root)
29 |     return -ENOMEM;
30 | for (i = 0; !files->name || files->name[0]; i++, files++) {
31 |     if (!files->name)
32 |         continue;
33 |
34 |     /* warn if it tries to conflict with the root inode */
35 |     if (unlikely(i == 1))
36 |         printk(KERN_WARNING "%s: %s_passed_in_a_files_array"
37 |             "with_an_index_of_1!\n", __func__,
38 |             s->s_type->name);
39 |
40 |     dentry = d_alloc_name(s->s_root, files->name);
41 |     if (!dentry)
42 |         return -ENOMEM;
43 |     inode = new_inode(s);
44 |     if (!inode) {
45 |         dput(dentry);
46 |         return -ENOMEM;
47 |     }
48 |     inode->i_mode = S_IFREG | files->mode;
49 |     simple_inode_init_ts(inode);
50 |     inode->i_fop = files->ops;
51 |     inode->i_ino = i;
52 |     d_make_persistent(dentry, inode);
53 |     dput(dentry);
54 | }
55 | return 0;
56 | }
57 | }

```

Листинг 33. simple_fill_super